

DEFINIENDO LA ARQUITECTURA PARA UN SISTEMA

R. A. Ramírez Gómez
7690-22-12700 Universidad Mariano Gálvez
Seminario de Tecnologías de Información
rramirezg18@miumg.edu.gt

Resumen

Elegir la arquitectura de una aplicación según su tipo es una decisión que afecta el diseño, el desarrollo y el mantenimiento de todo el sistema, por lo que es importante conocer los criterios que guían esta elección. Se investigó sobre los principales estilos arquitectónicos como el monolítico, el de N capas, el de microservicios y el dirigido por eventos, además de que se abordó cómo cada estilo se ajusta mejor a cierto tipo de dominio y de requisitos. También se explicó el papel de los atributos de calidad o requisitos no funcionales como criterio central para tomar esta decisión, apoyándose en el estándar ISO/IEC 25010. Se concluye que no existe una arquitectura mejor que otra de forma universal, sino que la elección depende del tipo de aplicación, de la complejidad del dominio y de los atributos de calidad que se prioricen en cada proyecto.

Palabras clave: arquitectura de software, estilos arquitectónicos, microservicios, atributos de calidad, requisitos no funcionales

Desarrollo del tema

La arquitectura de una aplicación es una de las primeras decisiones que se deben tomar durante el desarrollo de un sistema, ya que define la estructura general y afecta directamente el diseño, el desarrollo y el mantenimiento posterior. Microsoft (2025) explica que un estilo arquitectónico es una familia de arquitecturas que comparten ciertas características, y que aunque estos estilos no exigen el uso de tecnologías específicas, algunas tecnologías se adaptan mejor a determinados estilos, como es el caso de los contenedores con los microservicios.

Uno de los estilos más tradicionales es la arquitectura de N capas, en la cual la aplicación se divide en capas lógicas donde cada una tiene una responsabilidad específica y solo se comunica con las capas que están por debajo de ella. Microsoft (2025) señala que las capas típicas son la de presentación, la de lógica de negocio y la de acceso a datos, además de que este estilo es adecuado para aplicaciones empresariales tradicionales cuyo dominio es estable y cuya frecuencia de actualización es baja, aunque su estructura horizontal puede dificultar la introducción de cambios sin afectar varias partes de la aplicación.

En contraste, la arquitectura monolítica organiza toda la aplicación como un único componente que se despliega de forma completa y que utiliza una sola base de datos. Richardson (2024) explica que la principal ventaja del monolito es que todas las operaciones son locales, lo cual hace que las interacciones sean más simples de entender y más eficientes, además de que permite implementar transacciones de forma sencilla. Sin embargo, Richardson (2024) advierte que estas ventajas se van perdiendo a medida que la aplicación crece en tamaño y complejidad, ya que el componente único se vuelve difícil de entender y mantener, la autonomía de los equipos disminuye porque todos trabajan sobre el mismo código, y el proceso de despliegue se vuelve lento. Por esta razón, la arquitectu-

ra monolítica es una buena opción para aplicaciones pequeñas, con un dominio simple y equipos reducidos.

Entre el monolito y estilos más distribuidos existe la arquitectura Web-Queue-Worker, la cual está compuesta por un front-end web que atiende las peticiones de los usuarios, una cola de mensajes y un componente de trabajo en segundo plano que realiza las tareas pesadas o de larga duración (Microsoft, 2025). Según esta misma fuente, la comunicación entre el front-end y el componente de trabajo ocurre de forma asíncrona a través de la cola, y este estilo es ideal para aplicaciones con un dominio relativamente simple que tienen algunas tareas que consumen muchos recursos, además de que permite escalar el front-end y el componente de trabajo de forma independiente. Este estilo resulta útil cuando una aplicación no justifica la complejidad de los microservicios pero sí necesita separar el procesamiento pesado de la atención directa al usuario.

Cuando la aplicación crece y el dominio se vuelve más complejo, una alternativa es la arquitectura de microservicios, la cual descompone la aplicación en un conjunto de servicios pequeños y autónomos, donde cada uno implementa una única capacidad de negocio y gestiona su propia base de datos (Microsoft, 2025). Este estilo permite que los equipos trabajen de forma autónoma y que se realicen actualizaciones frecuentes con mayor velocidad de entrega, por lo que resulta adecuado para dominios complejos que requieren cambios constantes, aunque introduce una complejidad significativa en aspectos como el descubrimiento de servicios, la consistencia de los datos y la gestión del sistema distribuido, lo cual exige prácticas maduras de desarrollo y de DevOps (Microsoft, 2025).

Otro estilo relevante es la arquitectura dirigida por eventos, en la cual existen productores que generan flujos de eventos y consumidores que reaccionan a esos eventos casi en tiempo real, estando ambos desacoplados entre sí (Microsoft, 2025). Este estilo destaca en escenarios que requieren procesamiento en tiempo real con baja latencia, como soluciones de Internet de las cosas (IoT), sistemas de comercio financiero o aplicaciones que procesan grandes volúmenes de datos en streaming, aunque presenta desafíos relacionados con la entrega garantizada de los eventos, el orden de los mismos y la consistencia eventual entre los componentes distribuidos (Microsoft, 2025).

Cuando el tipo de aplicación gira en torno al procesamiento de grandes cantidades de información, existen estilos especializados. Microsoft (2025) describe la arquitectura de big data, la cual está orientada a la ingesta, el procesamiento y el análisis de datos que son demasiado grandes o complejos para los sistemas de bases de datos tradicionales, e incluye componentes para el almacenamiento de datos, el procesamiento por lotes para el análisis histórico y el procesamiento de flujos para obtener información en tiempo real. De forma similar, la arquitectura de big compute está pensada para cargas de trabajo que requieren cientos o miles de núcleos de procesamiento para operaciones intensivas de cómputo, como simulaciones, modelado de riesgo financiero o renderizado en tres dimensiones (Microsoft, 2025). Estos dos estilos muestran que el tipo de problema que se quiere resolver, ya sea el análisis de datos masivos o el cálculo intensivo, también determina la arquitectura más adecuada.

Un criterio central para elegir entre estos estilos es el conjunto de atributos de calidad, también conocidos como requisitos no funcionales, que definen cómo debe comportarse el sistema más allá de lo que debe hacer. La norma ISO/IEC (2011), a través del estándar 25010, define un modelo de calidad de producto compuesto por ocho características que son la adecuación funcional, la eficiencia de desempeño, la compatibilidad, la usabilidad, la fiabilidad, la seguridad, la mantenibilidad y la portabilidad. Estas características proporcionan una terminología común que permite especificar, medir y evaluar la calidad de un sistema, y sirven como referencia para comparar los requisitos de calidad de un proyecto.

La relación entre los atributos de calidad y la elección de la arquitectura es directa, ya que Sánchez Castro (2023) explica que los requisitos no funcionales establecen los criterios que permiten evaluar cómo debe funcionar una aplicación en un entorno productivo, y que al diseñar una solución es importante considerar la combinación de atributos que permitan obtener una aplicación efectiva. De este modo, si un proyecto prioriza la escalabilidad y la capacidad de actualización frecuente, un estilo como el de microservicios puede ser más adecuado, mientras que si prioriza la simplicidad y un bajo costo inicial, un monolito bien estructurado puede ser la mejor opción.

Esta relación se puede observar de forma más concreta al asociar cada atributo con el estilo que mejor lo favorece. Cuando el atributo prioritario es la mantenibilidad y la facilidad de despliegue independiente, los microservicios ofrecen una ventaja clara al permitir que cada servicio evolucione por separado (Microsoft, 2025). Si el atributo más importante es la eficiencia de desempeño en operaciones simples, un monolito puede resultar más eficiente porque todas sus operaciones son locales y no dependen de la red (Richardson, 2024). Cuando se prioriza la fiabilidad y la capacidad de respuesta en tiempo real ante eventos, la arquitectura dirigida por eventos es la más apropiada, mientras que si el atributo central es la portabilidad hacia entornos ya existentes, la arquitectura de N capas facilita la migración con pocos cambios (Microsoft, 2025). Este ejercicio de asociar atributos con estilos convierte una decisión que parece abstracta en un análisis más objetivo y guiado por las necesidades reales del proyecto.

Independientemente del estilo que se elija, existen ciertos desafíos que se deben tener en cuenta durante la decisión. Microsoft (2025) señala que la complejidad de la arquitectura debe corresponder con la complejidad del dominio, ya que un diseño demasiado simple puede degenerar en una estructura desordenada, mientras que uno demasiado complejo para un problema pequeño agrega dificultades innecesarias. Otros desafíos que se deben considerar son el uso de mensajería asíncrona, que mejora el desacoplamiento y la escalabilidad pero introduce el problema de la consistencia eventual, y la comunicación entre servicios, que en arquitecturas distribuidas puede generar problemas de latencia o congestión en la red (Microsoft, 2025). Tener presentes estos desafíos ayuda a que la elección del estilo sea realista y no solo teórica.

Finalmente, Microsoft (2025) recomienda que la elección del estilo arquitectónico se realice con la participación de los interesados del proyecto, comenzando por identificar la naturaleza del problema que se quiere resolver y definiendo los principales impulsores del negocio junto con los atributos de calidad que se deben priorizar. Esta misma fuente advierte que antes de elegir un estilo es esencial comprender sus principios y restricciones, ya que de lo contrario se corre el riesgo de crear un diseño que aparente seguir el estilo sin obtener realmente sus beneficios.

Observaciones y comentarios

Considero que muchas veces se elige la arquitectura de microservicios por ser la más popular o moderna, sin evaluar si el proyecto realmente la necesita, lo cual termina agregando una complejidad innecesaria. También pude notar que los atributos de calidad no siempre se definen desde el inicio del proyecto, aunque son el criterio más objetivo para decidir qué estilo arquitectónico conviene según el tipo de aplicación.

Conclusiones

1. La arquitectura orientada a microservicios mal desarrollada provoca alto costo y problemas de mantenimiento.

2. No existe una arquitectura mejor que otra de forma universal, sino que la elección depende del tipo de aplicación, de la complejidad del dominio y de la frecuencia con la que se necesite actualizar el sistema.
3. Los atributos de calidad o requisitos no funcionales son el criterio central para elegir un estilo arquitectónico, ya que definen cómo debe comportarse el sistema en un entorno real.
4. Elegir un estilo arquitectónico sin comprender sus principios y restricciones puede llevar a un diseño que aparenta seguir el estilo pero que no obtiene sus beneficios reales.

Referencias

- ISO/IEC. (2011). *ISO/IEC 25010:2011. Systems and software engineering — Systems and software Quality Requirements and Evaluation (SQuaRE) — System and software quality models*. International Organization for Standardization. <https://www.iso.org/standard/35733.html>
- Microsoft. (2025). *Architecture styles*. Azure Architecture Center. <https://learn.microsoft.com/en-us/azure/architecture/guide/architecture-styles/>
- Richardson, C. (2024). *Pattern: Monolithic architecture*. Microservices.io. <https://microservices.io/patterns/monolithic.html>
- Sánchez Castro, J. (2023). *Atributos de calidad en la arquitectura de software*. LinkedIn. <https://es.linkedin.com/pulse/atributos-de-calidad-en-la-arquitectura-software-castro-ramirez>

Código fuente: <https://github.com/rramirezg18/foros/tree/main/foro3>